
Unit
À UNIT
RECIFE

ESTRUTURA DE DADOS AVANÇADA





Já trabalhamos com a linguagem C.

Então, a gente tá craque, não é?



Que tal trabalharmos com Python, agora?

- Em Python, não há ponteiros no mesmo sentido que em linguagens como C ou C++.
- Mas, Python gerencia referências a objetos de maneira similar a ponteiros, o que pode causar confusão para programadores vindos de C.
- Em Python, não trabalhamos diretamente com endereços de memória, mas sim com referências a objetos.

Antes de começarmos -> Ponteiros

- Em Python, não há pont… …ntido que em linguagens como C ou C++.
- Mas, Pyth… …e maneira similar a ponteiros, … …ra programadores vindos de C.
- Em Python, n… …diretamente com endereços de memória, mas … …referências a objetos.

Quando atribuímos uma variável a outra, ambas passam a referenciar o mesmo objeto na memória. Isso pode levar a efeitos colaterais inesperados se não for bem compreendido.

Antes de começarmos -> Ponteiros

- Python trata objetos de forma diferente dependendo se são **imutáveis** (inteiros, strings, tuplas) ou **mutáveis** (listas, dicionários, conjuntos).
- Os argumentos são passados por **referência para objetos mutáveis** e por **cópia de referência para objetos imutáveis**.

```
a = 10
b = a
b += 5
print(a)  # 10 (inteiros são
imutáveis)

lista1 = [1, 2, 3]
lista2 = lista1
lista2.append(4)
print(lista1)  # [1, 2, 3, 4]
(listas são mutáveis)
```

Antes de começarmos -> Ponteiros

python™

```
class Node:
    def __init__(self, data):
        self.data = data
        self.children = []
```

Unit

Nós

```python
class Node:
    def __init__(self, data):
        self.data = data
        self.children = []
```

Definição da Classe Node:

O código começa definindo uma classe chamada Node. Essa classe representa um nó genérico, onde cada nó pode ter múltiplos filhos.

Nós

A classe Node tem dois elementos principais:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.children = []
```

Atributo data: Contém os dados que o nó armazena.

Nós

A classe Node tem dois elementos principais:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.children = []
```

Atributo children: Uma lista que armazena os filhos do nó.

Nós

Método __init__:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.children = []
```

Construtor da classe, é chamado automaticamente quando uma nova instância da classe Node é criada. Inicia o objeto recém-criado com certos atributos.

Nós

Método __init__:

Referência à própria instância da classe.

```
class Node:
    def __init__(self, data):
        self.data = dat
        self.children
```

Parâmetro que é passado para o método.

Nós

Método __init__:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.children = []
```

Valor passado no parâmetro

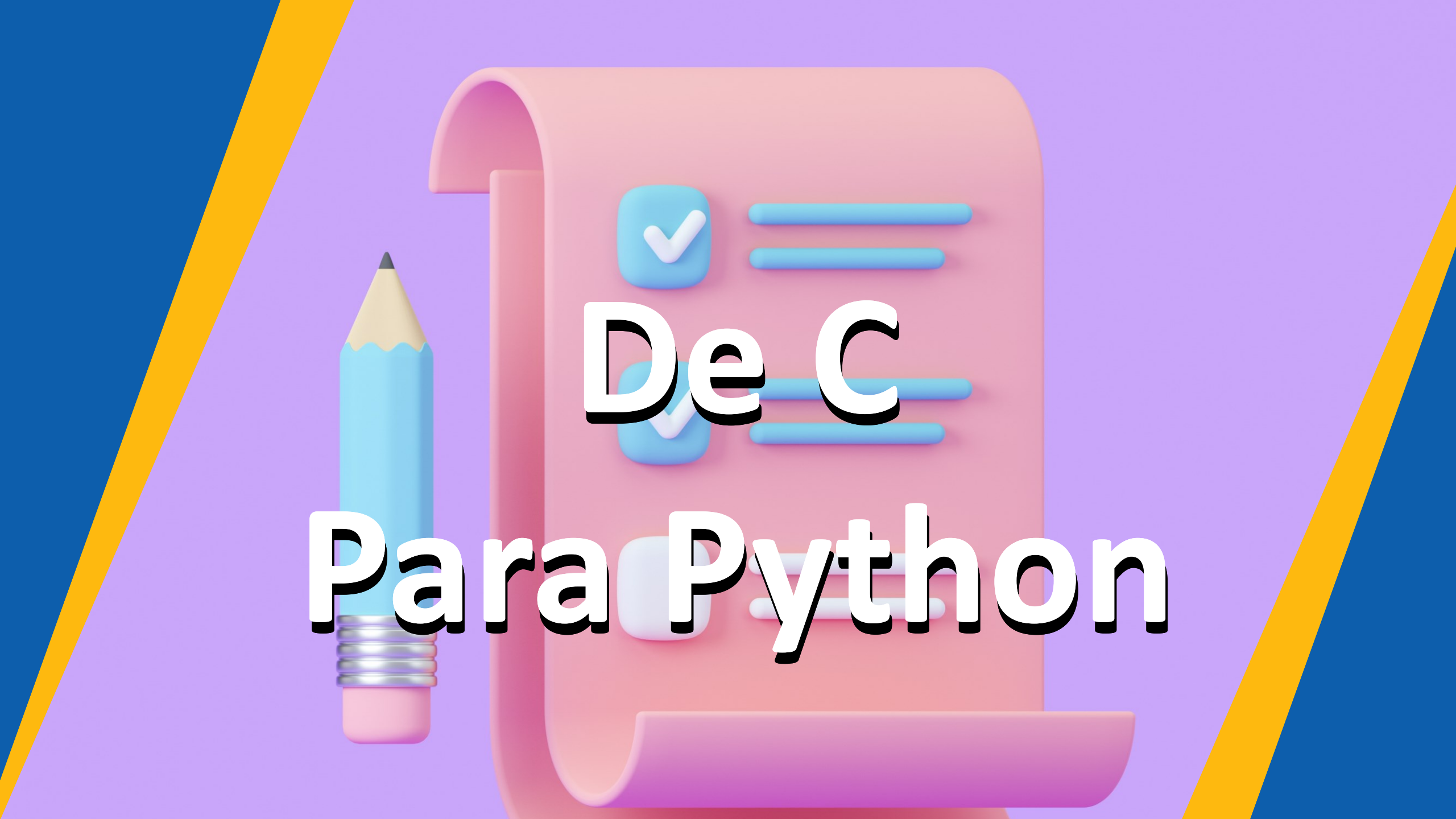
Nós

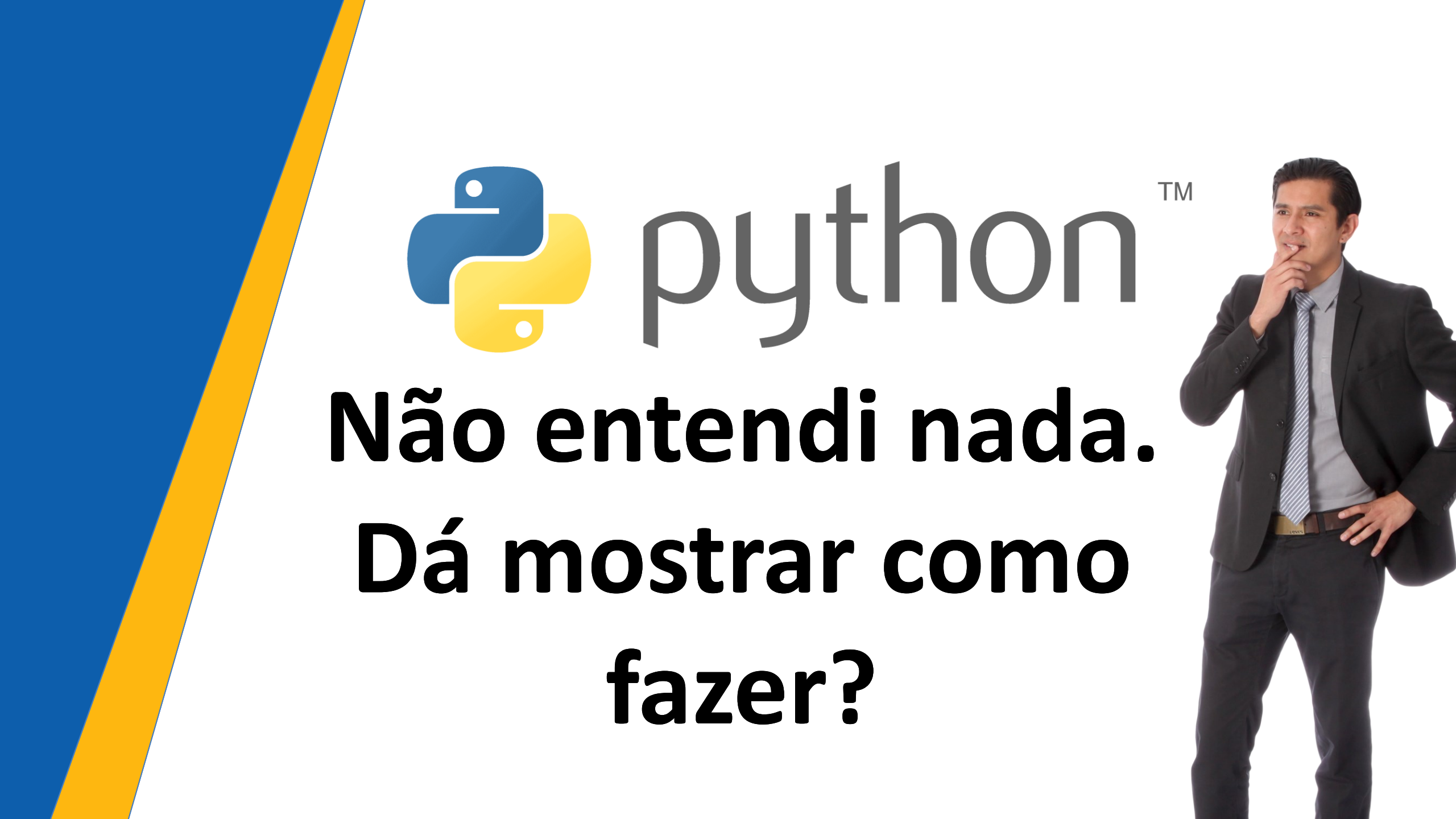
Método __init__:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.children = []
```

armazenará os filhos do nó

Nós


De C
Para Python


python™
Não entendi nada.
Dá mostrar como
fazer?



Antes de mostrar os códigos, vamos ver o que lembramos desses temas?

Unit

Listas, filas, pilhas e tabelas hash

python™

Tá, mas e agora?







Vamos
falar mais sobre
árvores?



Aplicações



Aplicações

Ontologias

python™

Aplicações

Ontologias

Forma de representar conhecimento em uma estrutura hierárquica, onde as relações entre conceitos podem ser representadas como **nós** e **arestas** em uma árvore. Cada nó pode representar um conceito, e seus filhos representam conceitos mais específicos ou subclasses desse conceito.

Aplicações

python™

- Ontologias
- Machine Learning

Aplicações

Ontologias

Machine Learning

Em Machine Learning, especialmente em tarefas de classificação hierárquica, é possível organizar classes e sub-classes em uma estrutura de árvore.

Aplicações

python™

- Ontologias
- Machine Learning
- PNL

Aplicações

Ontologias

Machine Learning

PNL

O parsing de uma frase em linguagem natural pode ser modelado como uma árvore sintática, onde cada nó da árvore representa um componente da frase (como sujeito, verbo, objeto).


Vamos
codar?

Aplicações

python™

Ontologia

```
class Node:
    def __init__(self, data):
        self.data = data  # O conceito ou o nome do nó (por exemplo, "Animal")
        self.children = []  # Lista de filhos, que serão subcategorias ou exemplos

# Criando os nós da ontologia
animal = Node("Animal")
mamifero = Node("Mamífero")
reptil = Node("Réptil")
cachorro = Node("Cachorro")
gato = Node("Gato")
cobra = Node("Cobra")
iguana = Node("Iguana")

# Estruturando a hierarquia
animal.children.append(mamifero)
animal.children.append(reptil)

mamifero.children.append(cachorro)
mamifero.children.append(gato)

reptil.children.append(cobra)
reptil.children.append(iguana)

# Imprimindo a ontologia
def print_ontologia(node, level=0):
    print("--" * level + node.data)
    for child in node.children:
        print_ontologia(child, level + 1)

print_ontologia(animal)
```

Aplicações

python™

```
class Node:
    def __init__(self, data):
        self.data = data  # Nome da
categoria ou do produto
        self.children = []  #
Subcategorias ou produtos específicos

# Criando os nós da árvore de
classificação
produtos = Node("Produtos")
eletronicos = Node("Eletrônicos")
informatica = Node("Informática")
computadores = Node("Computadores")
laptops = Node("Laptops")
smartphones = Node("Smartphones")

# Estruturando a hierarquia de
classificação
produtos.children.append(eletronicos)
eletronicos.children.append(informatica)
informatica.children.append(computadores)
computadores.children.append(laptops)

eletronicos.children.append(smartphones)

# Imprimindo a classificação hierárquica
def print_classificacao(node, level=0):
    print("--" * level + node.data)
    for child in node.children:
        print_classificacao(child, level
+ 1)

print_classificacao(produtos)
```

Machine Learning

Aplicações

```
class Node:
    def __init__(self, data):
        self.data = data  # A palavra ou a categoria sintática (ex: sujeito, verbo)
        self.children = []  # Lista de filhos (componentes da frase)

# Criando os nós da árvore sintática
sujeito = Node("O cachorro")
verbo = Node("comeu")
objeto = Node("o peixe")

# Criando a estrutura hierárquica
s = Node("S")  # Frase (S)
vp = Node("VP")  # Frase Verbal (VP)

# Montando a árvore
s.children.append(sujeito)  # Sujeito da frase
s.children.append(vp)  # Frase Verbal é parte da frase

vp.children.append(verbo)  # Verbo da frase
vp.children.append(objeto)  # Objeto da frase

# Imprimindo a árvore sintática
def print_tree(node, level=0):
    print("--" * level + node.data)
    for child in node.children:
        print_tree(child, level + 1)

print_tree(s)
```

PNL

Aplicações

```
class Node:
    def __init__(self, data):
        self.data = data  # A palavra ou a categoria sintática (ex: sujeito, verbo)
        self.children = []  # Lista de filhos (componentes da frase)

# Criando os nós da árvore sintática
sujeito = Node("O cachorro")
verbo = Node("comeu")
objeto = Node("o peixe")

# Criando a estrutura hierárquica
s = Node("S")  # Frase (S)
vp = Node("VP")  # Frase Verbal (VP)

# Montando a árvore
s.children.append(sujeito)  # Sujeito da frase
s.children.append(vp)  # Frase Verbal é parte da frase

vp.children.append(verbo)  # Verbo da frase
vp.children.append(objeto)  # Objeto da frase

# Imprimindo a árvore sintática
def print_tree(node, level=0):
    print("--" * level + node.data)
    for child in node.children:
        print_tree(child, level + 1)

print_tree(s)
```

PNL



Implemente uma árvore para representar:

Empresa
|-- Departamento1
| |-- Funcionário1
| |-- Funcionário2
|-- Departamento2
  |-- Funcionário3

Tarefinha



Implemente uma árvore para representar:

```
Empresa
|-- Departamento1
|    |-- Funcionário1
|    |-- Funcionário2
|-- Departamento2
     |-- Funcionário3
```

Modifique a estrutura para contar o número total de nós.

Tarefinha



Implemente uma árvore para representar:

Empresa
|-- Departamento1
| |-- Funcionário1
| |-- Funcionário2
|-- Departamento2
  |-- Funcionário3

Adicione uma funcionalidade para buscar um nó por nome.

Tarefinha